Neuronal Redundancy in Convolutional Networks: Measuring Parameters, Representations, and Functional Sensitivity

Independent exploratory research project

27th October 2026

Abstract

This project studies redundancy in the internal representations of convolutional neural networks trained to classify handwritten MNIST digits. The overarching question is to what extent multiple units exhibit similar activity, explore a low-dimensional representation space, or can be removed without materially changing the model's predictions. To distinguish these notions, the pipeline computes six complementary measures across three spaces: parameter similarity (P_L, P_C), activation geometry (R_L, R_C), and functional sensitivity (F_L, F_C). The measures are tracked during training on fixed samples and supplemented by a final summary on a separate test sample. The planned screening is exploratory: the catalogue contains 32 configurations with two seeds per condition, but the full catalogue has not yet been run. No consolidated scientific findings are claimed. The longer-term aim is to investigate statistical relationships between the measures and observable properties of learning, without treating a correlation or low sensitivity as a causal explanation of the computation learned by the network.

Keywords: neural redundancy; representations; effective dimensionality; ablation; MNIST; Model Zoos

1 Introduction

Neural networks may contain units whose responses are correlated, whose activity can be reconstructed from other units, or whose removal has little effect on the network's output. These situations are often grouped under the term *redundancy*, but they are not equivalent. Two units may have similar activations without being functionally interchangeable; conversely, a distributed direction across several units may be of little functional importance even when no individual unit is. Work on redundant units distinguishes units that can be removed with little functional effect from units whose activity is linearly redundant [1]. Comparing neural representations is also a methodological problem in its own right: results depend on the object being compared and on the invariances of the chosen measure [2].

This work establishes an empirical protocol for examining these notions in a controlled setting. It measures the geometry of a layer's weights, the responses of its units to images, and the effect of interventions on those responses on the network's predictions. The central intuition is to compare three levels of analysis rather than infer redundancy from a single matrix: similar weights may produce different activity, and redundant activity alone does not guarantee low functional importance.

The project draws on the MNIST model dataset associated with the Model Zoos repository, which provides models and data generated under systematic factors [3]. The experiment described here does not simply compare pre-trained checkpoints: the code loads a cached MNIST dataset and trains new CNNs under specified configurations. Work to date has focused on the run-generation pipeline, the measures, and their temporal tracking. The configured experiments remain exploratory, and relationships between measures have not yet been statistically analysed.

2 Research question and conceptual framework

Here, *neural redundancy* refers to a situation in which multiple units carry overlapping information or influence under an explicitly defined criterion. This operational definition covers three distinct questions:

1. Do the incoming or outgoing weights of two units point in similar directions?
2. Are unit responses correlated, and does their collective activity occupy few directions of variation?
3. Does removing a unit or subspace alter the network's output distributions?

The six principal measures are organised around these questions: P_L and P_C concern parameter space; R_L and R_C, representation space; and F_L and F_C, function space as observed on a set of images. The subscripts L and C denote local (a pair of units or a single unit) and collective (a spectrum or subspace) analyses, respectively. They do not imply that values from different families are directly comparable: their definitions, scales, and inputs differ.

The project does not assume that these indicators alone reveal *what the network computes* in a semantic sense. They quantify geometric properties and behavioural sensitivities. Interpreting the computation will require further analyses, for example, assessing stability across seeds, associations with performance, and targeted interventions on classes or examples.

3 Related work

Model zoos have been proposed as structured populations of networks for studying model diversity, parameters, and training trajectories [3]. This perspective motivates the population-level analysis envisaged here: beyond a single network, runs from different conditions and seeds may help establish which properties recur and which depend on training.

Methods for representation similarity have highlighted that a valid comparison depends on the transformations to which one wishes to be invariant. Kornblith et al. present CKA as one approach to comparing representations and discuss limitations of other multivariate measures [2]. This project instead uses simpler, within-layer measures: Pearson correlations and effective dimensionality, rather than CKA between two networks. R_L should therefore not be interpreted as a measure of inter-model similarity.

Finally, work on so-called "frivolous" units distinguishes removal without substantial functional loss from linear redundancy in responses [1]. This motivates complementing activity measures with functional ablations. The current protocol does not reproduce that taxonomy or its decision criteria: it provides continuous scores, primarily distribution divergences, which must be interpreted comparatively and without a universal threshold.

4 Methods

4.1 Data and model provenance

The pipeline uses the tensor-form MNIST dataset cached with Model Zoos. This provenance is associated with *Model Zoos: A Dataset of Diverse Populations of Neural Network Models* [3]. In the current protocol, the training split is used for optimisation, while validation and test splits are reserved for monitoring and evaluation. A deterministic 40% subset of the training split is selected using subsampling seed 2026. Labels correspond to the ten MNIST digit classes.

The provenance of the dataset should be distinguished from the object actually trained: experimental runs train new models using the architectures and hyperparameters in the YAML

plan; they are not simply measurements applied to the zoo's pre-trained weights. The measurement code automatically selects the final linear hidden layer and its non-linearity. In the active CNN architecture, this representation has width 20 and precedes the ten-output classification layer.

4.2 Configured experimental plan

The active plan fixes the architecture to CNN, initialisation to `uniform`, batch size to 128, training duration to 20 epochs, and execution device to CPU. It crosses two activations (tanh, ReLU), two dropout levels (0 and 0.15), two optimisers (Adam, SGD), two learning rates (10^{-3} and 10^{-2}), and two model seeds (11 and 22). Weight decay is fixed at 10^{-4}; momentum is fixed at 0.9 and Nesterov is disabled. The Cartesian product gives 16 hyperparameter combinations, with two seeds yielding 32 planned runs.

The 32-run catalogue is the declarative plan. The current launch configuration sets `run_limit: 0`, which means no catalogue limit and therefore permits all 32 runs to execute. The planned screening is a single, unified launch under the current measurement pipeline. The project remains exploratory, rather than a statistically powered design for confirmatory conclusions. Other options supported by the code are not part of the active plan.

Table 1: Experiment configuration currently defined in the YAML files.

Item	Active configuration
Data	Cached tensor-form MNIST; 40% of training split
Measured architecture	CNN; final hidden layer of width 20
Activations	tanh, ReLU
Dropout	0; 0.15
Optimisers	Adam, SGD
Learning rates	0.001; 0.01
Epochs / batch size / device	20 / 128 / CPU
Seeds per condition	11 and 22
Catalogue runs / launch limit	32 / none (0)
Measurement cadence	every epoch; 500 training and validation examples
F_C dimensions / random controls	$k = 1, \ldots, 20$ / 50 per dimension

4.3 Training and evaluation protocol

Each run initialises a CNN using its corresponding seed and trains it on the deterministic MNIST subset. Training metrics are recorded at every epoch, and validation is evaluated after every epoch. The temporal configuration requests measurements at every epoch and selects a fixed, ordered sample of 500 examples from each split. Thus, within a run, the same examples are used across epochs, making trajectories more directly comparable.

At each measured epoch, the current checkpoint's weights and activations are used to compute the six measure families: parameter measures are recomputed from the current weights, while activation and functional measures are computed separately on the fixed training and validation samples. Diagnostic evaluations on those samples also record loss and accuracy at every epoch. The full test split is evaluated only at the pre-selected final checkpoint, to limit repeated exposure to results from the held-out set.

The final-summary pipeline also measures on D_{ref}, consisting of the first 500 examples of the test split in the recorded order, and checks final accuracy consistency. This final sample is distinct from the temporal training and validation samples. Sampling policies are recorded with the results for traceability.

4.4 Measure definitions

Let $d = 20$ be the number of units in the final hidden layer. For a set of n images, let $A \in \mathbb{R}^{n \times d}$ be the matrix whose row m contains the post-non-linearity activation vector $a(x_m)$. Each column therefore corresponds to one unit observed on the same examples. Let $W_{\text{in}} \in \mathbb{R}^{d \times q}$ be the incoming weights of the hidden linear layer, with one row per unit, and let $W_{\text{out}} \in \mathbb{R}^{d \times 10}$ be the transposed outgoing weights, also with one row per unit.

Local parameter similarity (P_L). For units i, j, the code separately computes the cosine similarity between their incoming weight vectors and between their outgoing weight vectors, then takes the mean:

$$P_L(i,j) = \frac{1}{2}\left(\frac{W_{\text{in},i} \cdot W_{\text{in},j}}{\|W_{\text{in},i}\|_2 \|W_{\text{in},j}\|_2} + \frac{W_{\text{out},i} \cdot W_{\text{out},j}}{\|W_{\text{out},i}\|_2 \|W_{\text{out},j}\|_2}\right).$$

The P_L matrix describes directional alignment of parameters associated with units, both upstream and downstream. It is not a direct measure of similarity between computed functions: non-linearities, observed activations, and interactions with the rest of the network also matter.

Effective dimensionality of weights (P_C). Each row of W_{in} is normalised to unit norm, then the singular values s_i of the normalised matrix are computed. The score is the spectral-energy participation ratio:

$$P_C = \frac{\left(\sum_i s_i^2\right)^2}{\sum_i s_i^4}.$$

It summarises the effective number of spectral directions supporting the incoming weight vectors: a concentrated spectrum gives a lower value, while a more uniformly distributed spectrum gives a higher value. It is not the exact rank. Row normalisation means that P_C describes incoming-weight directions rather than their magnitudes. This is the definition implemented by the current code.

Local activation correlations (R_L). For each pair of units, $R_L(i,j)$ is the Pearson correlation between the corresponding columns of A. A value near $+1$ indicates linearly aligned variation in the sample; a value near -1 indicates opposing variation; a value near zero indicates no marked linear association, without ruling out non-linear dependence. Pairs involving a constant unit are undefined and recorded as such, rather than interpreted as zero correlation.

Effective dimensionality of representations (R_C). The code centres the columns of A, computes the singular values s_i of the centred matrix, and forms covariance eigenvalues proportional to $\lambda_i = s_i^2$. The score is:

$$R_C = \frac{\left(\sum_i \lambda_i\right)^2}{\sum_i \lambda_i^2}.$$

This participation-ratio form is a continuous measure of effective dimensionality from the covariance spectrum [4]. If variance is distributed uniformly across r components and is zero elsewhere, the score is r; with one dominant component and a tail, the score remains continuous. A low value is consistent with low-dimensional collective activity, but does not by itself show that units are functionally interchangeable.

Local functional sensitivity (F_L). The network computes its original log-probabilities $\log p_\theta(\cdot \mid x)$. For each unit j, the code sets its post-nonlinearity activation to zero, without retraining the network, and then computes the mean Kullback–Leibler divergence between the original and perturbed outputs:

$$F_L(j) = \frac{1}{n} \sum_{m=1}^{n} D_{\mathrm{KL}}\Big(p_\theta(\cdot \mid x_m) \,\Big\|\, p_{\theta,\, a_j \leftarrow 0}(\cdot \mid x_m)\Big) .$$

The divergence is computed from distributions over the ten classes and is expressed in nats per example. A score of zero means that this ablation leaves the distributions unchanged on the sample; a higher score indicates a larger change. This is functional importance local to this model, layer, sample, and intervention, not a universal measure of utility. Changes in label-based NLL and accuracy are secondary diagnostics; they do not replace KL.

Collective functional sensitivity (F_C). A PCA is computed from activations centred on the current sample. For dimension k, the code compares two types of orthonormal subspace: (i) the k lowest-variance directions, corresponding to principal components with the smallest eigenvalues; and (ii) 50 deterministic random subspaces of dimension k, used as controls. For each subspace D and activation vector a, the intervention removes the projection of the centred variation while preserving the reference mean μ:

$$a_{\text{intervened}} = \mu + (a - \mu) - (a - \mu) D^{\mathsf{T}} D.$$

The $F_C(k)$ score is the same mean KL between original and intervened predictions as for F_L. The curve therefore compares the functional effect of removing a low-variance subspace with that of same-dimensional random subspaces. If the former is lower than the control distribution, this would be consistent with lower sensitivity to low-variance directions under the conditions evaluated; it would not establish a general causal relationship between variance and importance.

Remaining effective dimensionality after removing low-variance components. Let the activation covariance eigenvalues be ordered in descending order,

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_{20} \geq 0.$$

The pipeline removes the k lowest-variance principal components (the final k eigenvalues), then recomputes the participation ratio using the remaining $20 - k$ eigenvalues:

$$R_{C,\text{remaining}}(k) = \frac{\left(\sum_{i=1}^{20-k} \lambda_i\right)^2}{\sum_{i=1}^{20-k} \lambda_i^2}, \qquad 0 \leq k < 20.$$

This curve is a structural summary of the activity spectrum, not a functional measure: it indicates the effective dimensionality that remains, whereas $F_C(k)$ measures changes to the outputs after intervention. In the plot, $R_{C,\text{remaining}}(k)$ is shown with dotted lines and read against the right-hand vertical axis. At $k = 20$, the remaining spectrum is empty and the ratio is undefined by the formula; the code conventionally sets $R_{C,\text{remaining}}(20) = 0$.

The current launch configuration includes dimensions $k = 1, \ldots, 20$. The 50 draws provide an empirical control that depends on the random-direction generation seed; their dispersion should be retained in subsequent analyses. At $k = 20$, the scores remove the entire centred representation space and should be read as the limiting case of this intervention, not as an ordinary point.

Table 2: The six measures, their objects, and a cautious interpretation.

Measure	Computed object	Returned summary	Question explored
P_L	Incoming and outgoing weight directions	Pairwise cosine matrix	Are parameters associated with units aligned?
P_C	Spectrum of normalised incoming weights	Participation ratio	How many parameter directions contribute effectively?
R_L	Post-nonlinearity activations on fixed images	Pearson matrix	Which units covary linearly on these examples?
R_C	Covariance of centred activations	Participation ratio	Does activity vary within a reduced effective subspace?
F_L	Zeroing one unit at a time	Per-unit KL vector	Does the output depend on this individual unit?
F_C	Centred removal of a low-variance or random subspace	KL as a function of k and controls	Are low-variance directions less sensitive than same-dimensional random directions?

4.5 When are the measures computed?

Temporal mode is enabled in the current configuration with a cadence of one epoch. At each of the 20 epochs, P_L and P_C are computed from the checkpoint weights, then R_L, R_C, F_L, and F_C from activations and outputs on the 500 fixed training examples and 500 fixed validation examples. The temporal training sample is drawn from the training subset used for optimisation; it is therefore not an independent dataset. Validation and test serve distinct purposes.

After training, the pipeline also produces a final summary on the fixed test D_{ref}. These time series and the final summary should not be conflated: they concern different splits and serve, respectively, to track change over time and to provide a final evaluation. Generating plots and dashboards is an inspection tool; it is not a statistical analysis of the hypotheses.

5 Current status and planned analyses

At the time of this version, the repository contains a reproducible experiment catalogue, configurable and resumable training, deterministic sample selection, computation of the six measure families, temporal recording, and dashboards. The implemented definitions have evolved from the initial guide: P_C uses the spectral-energy participation ratio, consistently with R_C, and F_C applies a centred PCA projection that preserves the activation mean. This article describes the current conventions.

The versioned launcher configuration selects the full catalogue and creates a separate timestamped output folder for each launch. The screening results will be audited for training quality and measurement completeness before interpretation. Historical development outputs are excluded from the planned screening analysis.

The planned follow-up analysis will examine associations among the six measures and their evolution across epochs, as well as links with loss and accuracy. It must account for the factorial design, seeds, and temporal dependence: successive epochs within a run are not independent replicates. Runs within a condition also share a configuration; seeds should be treated as limited replications within that condition, not as fully independent observations of it. Descriptive analyses, uncertainty intervals, and appropriately chosen statistical models will be needed before claiming robust relationships. Any attempt to infer what computation the network performs will require further targeted analyses.

6 Limitations

The current protocol concerns MNIST and a fixed CNN architecture with a final hidden representation of only 20 units. Results will therefore be limited to this model, dataset, and set of tested hyperparameters. The current grid varies several factors simultaneously; effects and interactions must be estimated cautiously, and two seeds per condition do not guarantee a stable estimate of variability. The configured catalogue and the runs actually launched are also distinct quantities.

The measures depend on layer choice, reference examples, and parameterisation. P_L does not directly account for activations; R_L detects only pairwise linear association; R_C depends on an empirical spectrum estimated from a finite sample; and F_L and F_C measure interventions without retraining, not performance after adaptation. In particular, F_C concerns directions in activation space, whereas P_C concerns directions of incoming weights: similar values do not imply that they describe the same subspaces.

Finally, constant units are identified only on the measured sample. A unit that is constant on these examples cannot be declared globally inactive without further checks. Redundancy scores are no substitute for evaluation on more data or robustness checks across datasets, architectures, and seeds.

7 Conclusion

This project establishes a measurable framework for studying different facets of neural redundancy in CNNs trained on MNIST. Its current contribution is primarily methodological: bringing together parameter geometry, activation structure, and functional sensitivity, while tracking these quantities on fixed samples during training. The configured screening is a starting point for exploration and does not yet justify conclusions about what the network computes. The next scientific step is to complete the planned runs, quantify uncertainty, and test whether observed relationships between measures are reproducible and informative.

References

[1] Stephen Casper, Xavier Boix, Vanessa D'Amario, Ling Guo, Martin Schrimpf, Kasper Vinken, and Gabriel Kreiman. Frivolous units: Wider networks are not really that wide. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 6921–6929, 2021. doi: 10.1609/aaai.v35i8.16853. URL https://ojs.aaai.org/index.php/AAAI/article/view/16853.

[2] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3519–3529, 2019. URL https://proceedings.mlr.press/v97/kornblith19a.html.

[3] Konstantin Schürholt, Diyar Taskiran, Boris Knyazev, Xavier Giró-i Nieto, and Damian Borth. Model zoos: A dataset of diverse populations of neural network models. In *Advances in Neural Information Processing Systems*, volume 35, 2022. doi: 10.52202/068431-2763. URL https://arxiv.org/abs/2209.14764.

[4] Jörn Diedrichsen, Tobias Wiestler, and Naveed Ejaz. A multivariate method to determine the dimensionality of neural representation from population activity. *NeuroImage*, 76:225–235, 2013. doi: 10.1016/j.neuroimage.2013.02.062. URL https://doi.org/10.1016/j.neuroimage.2013.02.062.